

PRACTICE EXERCISES OF THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

Student's name: Văn Nhật Tân

Student code: 24521587

PRACTICE EXERCISE #3:

USING INTERRUPT

1. Sử dụng Timer và UART của vi điều khiển 8051, thiết kế mạch đồng hồ đếm giờ chính xác đến 1% giây (Từ 00.00 – 99.99), thiết kế các nút bấm có chức năng:

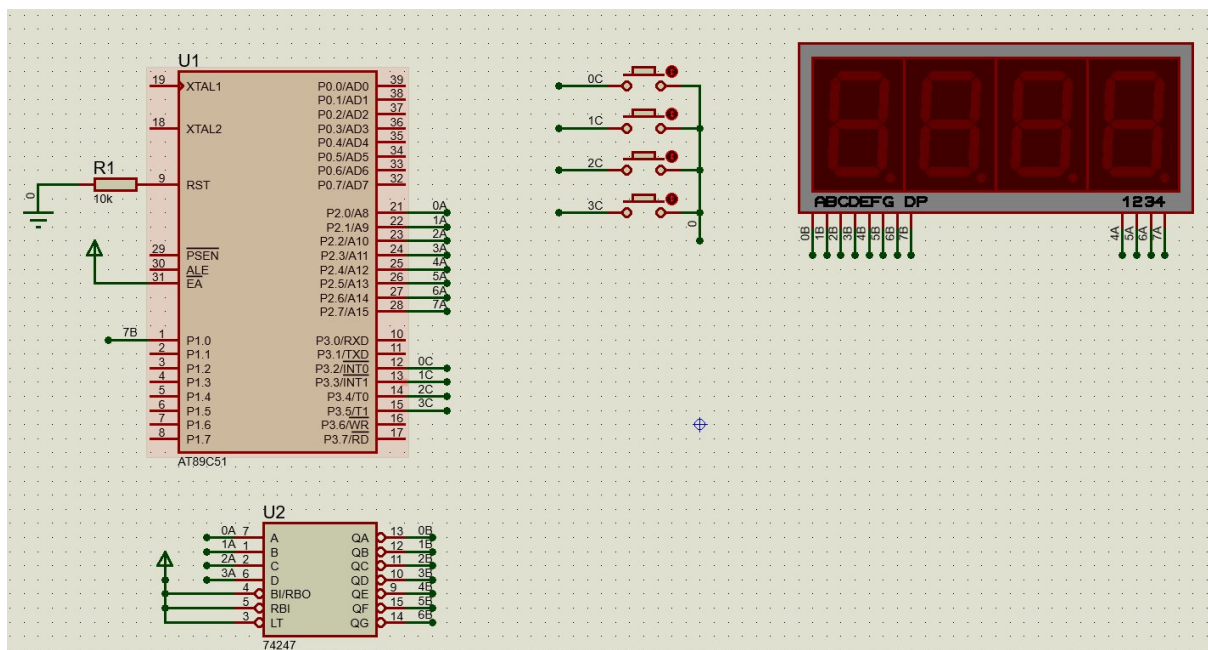
Nút A: Tạm dừng / Tiếp tục.

Nút B: Reset về 00.00s.

Nút C: Tăng giá trị đếm lên 1s.

Nút D: Giảm giá trị đếm đi 1s.

- a. Hình ảnh thiết kế.



- b. Nguyên lý hoạt động mạch:

Linh kiện:

- AT89C51.
- 74247: Chuyển đổi BCD sang hiển thị LED 7 đoạn
 - Chân A-D: Nhận giá trị BCD, A là bit LSB, D là MSB

- Chân BI/RBO: Xóa hiển thị. Nối với GND thực hiện tắt LED với mọi đầu vào, nối với VCC vô hiệu hóa chức năng.
 - Chân RBI: Lược hiển thị số 0 vô nghĩa. Nối với GND (vd: 500 à 500; 005 à 5), nối với VCC vô hiệu hóa chức năng.
 - Chân LT: Dùng để kiểm tra các LED có hoạt động đúng chức năng hay không. Nối GND thực hiện kiểm tra LED (Hiển thị số 8), nối với VCC vô hiệu hóa chức năng.
- 7SEG – MPX4 – CA: LED 7 đoạn 4 số
- Chân A-G nối với các chân QA-QG của 74247. (Tích cực mức thấp)
 - Chân 1-4: Chọn LED muốn hiển thị. (Tích cực mức cao)
 - Chân DP: Hiển thị các dấu chấm.
- Các nút bấm:
- Nút A (P3.2): Tạm dừng / Tiếp tục.
 - Nút B (P3.3): Đặt lại về 0.
 - Nút C (P3.4): Tăng giá trị đếm lên 1s.
 - Nút D (P3.5): Giảm giá trị đếm đi 1s.

c. Source Code

- Hệ thống sử dụng Timer 0 ở Chế độ 1 (16-bit) để tạo ngắt mỗi 10ms (tương đương 0.01s):
 - Với thạch anh 11.0592MHz, mỗi chu kỳ máy là $1.085\mu s$. Để có 10ms, ta cần nạp giá trị: $65536 - 9216 = 56320$ (tương đương DC00H).
 - Mỗi khi ngắt xảy ra, biến đếm phần trăm giây (R2) tăng lên. Khi R2 = 100, biến đếm giây (R1) sẽ tăng thêm 1 đơn vị.

Assembly	Giải thích
ORG 0100H LJMP START ORG 0003H LJMP INT_0_ISR ORG 000BH LJMP TIMER_0_ISR	ORG 0000H và LJMP START: Thiết lập địa chỉ bắt đầu của chương trình và nhảy đến hàm khởi tạo chính. ORG 0003H và LJMP INT_0_ISR: Khai báo vector ngắt

<pre> ORG 0013H LJMP INT_1_ISR SEC EQU R1 MSEC EQU R2 RUN_FLAG BIT 00H </pre>	ngoài 0 và lệnh nhảy đến chương trình đến hàm phục vụ ngắt. ORG 0013H và LJMP INT_1_ISR: Khai báo vector ngắt ngoài 1 và lệnh nhảy đến chương trình đến hàm phục vụ ngắt. SEC EQU R1 và MSEC EQU R2: R1 lưu giá trị giây, R2 lưu giá trị phần trăm giây. ORG 000BH và LJMP TIMER_0_ISR: Khai báo vector ngắt tràn bộ định thời và lệnh nhảy đến chương trình đến hàm xử lý timer. RUN_FLAG BIT 00H: Cờ trạng thái: 1 = Đang chạy, 0 = Tạm dừng.
<pre> START: SETB TCON.0 SETB TCON.2 MOV IE, #87H MOV IP, #05H MOV TMOD, #01H MOV TH0, #0DCH MOV TL0, #00H CLR RUN_FLAG CLR TR0 MOV SEC, #0 MOV MSEC, #0 MOV P3, #0FFH MOV P1, #0FFH MOV P2, #00H </pre>	SETB TCON.0, SETB TCON.2, MOV IE, #87H, MOV IP, #05H: Cấu hình và phân mức ưu tiên Ngắt. • TCON.0, TCON.2: Cấu hình Ngắt ngoài 0 và 1 kích hoạt mức thấp. • MOV IE, #87H: Mở ngắt toàn cục, ngắt Timer 0 và hai ngắt ngoài. • MOV IP, #05H: Đặt ưu tiên cao nhất cho hai ngắt ngoài để nút bấm luôn phản hồi tức thời. MOV TMOD, #01H, MOV TH0, #0DCH, MOV TL0, #00H: Cấu hình Timer 0. • MOV TMOD, #01H: Đặt Timer 0 ở Chế độ 1 (16-bit).

	• TH0, TL0: Nạp giá trị ban đầu là 0DCH. Với thạch anh 11.0592MHz, Timer đếm thêm 9261 xung (10ms) thì tràn. CLR RUN_FLAG, CLR TR0, MOV SEC, #0, MOV MSEC, #0: Khởi tạo biến và trạng thái. • CLR RUN_FLAG: Đặt trạng thái ban đầu là Tạm dừng. • CLR TR0: Chưa cho Timer 0 đếm thời gian. • MOV SEC, MSEC: Đặt giá trị Giây và Phần trăm giây về 00.00. MOV P3, #0FFH, MOV P1, #0FFH, MOV P2, #00H: Thiết lập phần cứng. • MOV P3, #0FFH: Kéo toàn bộ P3 lên mức 1 để thiết lập làm chân Input nhận tín hiệu nút bấm. • MOV P1, #0FFH: Tắt dấu chấm DP. • MOV P2, #00H: Đưa toàn bộ P2 về 0 để tắt hoàn toàn 4 LED ban đầu.
LOOP: LCALL CHECK_BTN_C LCALL CHECK_BTN_D LCALL DISPLAY SJMP LOOP	LCALL CHECK_BTN_C, LCALL CHECK_BTN_D, LCALL DISPLAY, SJMP LOOP: • Liên tục gọi các chương trình con để kiểm tra trạng thái nút

<pre> INT_0_ISR: CPL RUN_FLAG JB RUN_FLAG, START_TMR CLR TR0 RETI START_TMR: SETB TR0 RETI INT_1_ISR: MOV SEC, #0 MOV MSEC, #0 RETI TIMER_0_ISR: PUSH ACC PUSH PSW CLR TR0 MOV TH0, #0DCH MOV TL0, #00H SETB TR0 JNB RUN_FLAG, EXIT_ISR INC MSEC MOV A, MSEC CJNE A, #100, EXIT_ISR MOV MSEC, #0 INC SEC MOV A, SEC CJNE A, #100, EXIT_ISR MOV SEC, #0 </pre>	nhấn C, nút D và hàm quét hiển thị 4 LED 7 đoạn. • INT_0_ISR, CPL RUN_FLAG, JB RUN_FLAG, ...: Xử lý Ngắt ngoài 0 (Chức năng Tạm dừng / Tiếp tục). • CPL RUN_FLAG: Đảo ngược trạng thái của cờ chạy (ví dụ từ 0 sang 1, hoặc từ 1 về 0). • Lệnh JB kiểm tra: Nếu cờ này bằng 1, chương trình nhảy đến START_TMR để bật lại Timer 0 giúp đồng hồ chạy tiếp. Ngược lại nếu cờ bằng 0, vi điều khiển thực thi lệnh CLR TR0 để dừng Timer, làm đồng hồ tạm dừng đếm thời gian. INT_1_ISR, MOV SEC, #0, MOV MSEC, #0: Xử lý Ngắt ngoài 1 (Chức năng Reset). • Hàm này được gọi khi người dùng nhấn nút Reset (Nút B). • Ép giá trị các biến đếm Giây và Phần trăm giây trở về giá trị 00.00. TIMER_0_ISR, PUSH ACC, PUSH PSW ... POP PSW, POP ACC: Chống mất dữ liệu thanh ghi khi đang xử lý ngắt và Timer. CLR TR0, MOV TH0, #0DCH, MOV TL0, #00H, SETB TR0:
---	---

<pre> EXIT_ISR: POP PSW POP ACC RETI </pre>	Nạp lại thời gian cho Timer 0. • Tạm dừng Timer (CLR TR0) để nạp lại giá trị đếm một cách an toàn. • Nạp giá trị DC00H (dành cho thạch anh 11.0592MHz) để Timer đếm đúng 10ms cho lần ngắt tiếp theo. • Khởi động lại Timer (SETB TR0). JNB RUN_FLAG, EXIT_ISR: Kiểm tra điều kiện đếm giờ. • Nếu cờ RUN_FLAG đang là 0 lệnh này sẽ điều hướng chương trình nhảy thẳng đến cuối hàm (EXIT_ISR) để thoát ngắt, bỏ qua việc cộng dồn thời gian ở bên dưới. INC MSEC, CJNE A, #100, EXIT_ISR, INC SEC: • Tăng biến đếm phần trăm giây (MSEC). Lệnh CJNE kiểm tra nếu MSEC chưa bằng 100 thì thoát ngắt. • Tương tự, lệnh CJNE bên dưới kiểm tra nếu Giây chạm mốc 100 thì vòng ngược lại về 0.
<pre> DISPLAY: MOV A, MSEC MOV B, #10 DIV AB </pre>	MOV A, MSEC, MOV B, #10, DIV AB: • Lấy giá trị biến đếm (MSEC hoặc SEC) chia cho 10.

<pre> MOV A, B ANL A, #0FH ORL A, #10000000B MOV P2, A SETB P1.0 LCALL DELAY_LED MOV P2, #00H MOV A, MSEC MOV B, #10 DIV AB ANL A, #0FH ORL A, #01000000B MOV P2, A SETB P1.0 LCALL DELAY_LED MOV P2, #00H MOV A, SEC MOV B, #10 DIV AB MOV A, B ANL A, #0FH ORL A, #00100000B MOV P2, A CLR P1.0 LCALL DELAY_LED MOV P2, #00H MOV A, SEC MOV B, #10 DIV AB ANL A, #0FH ORL A, #00010000B MOV P2, A </pre>	• Kết quả phép chia cho ta hai phần: Phần dư lưu ở thanh ghi B (chính là chữ số hàng đơn vị) và Phần thương lưu ở thanh ghi A (chính là chữ số hàng chục). ANL A, #0FH: Làm sạch dữ liệu BCD (4 bit thấp). • Lệnh AND logic với #0FH (00001111B) có tác dụng giữ nguyên 4 bit thấp và xóa 4 bit cao thành mức 0. ORL A, #10000000B (và các lệnh ORL tương tự): Ghép tín hiệu quét chọn LED. • Lệnh OR logic kết hợp mã BCD với tín hiệu chọn vị trí LED thành một byte duy nhất để xuất ra Port 2. SETB P1.0 và CLR P1.0: Điều khiển dấu chấm thập phân (DP). • SETB P1.0: Đưa chân P1.0 lên mức 1 để TẮT dấu chấm tại các con LED 1, 3 và 4. • CLR P1.0: Kéo chân P1.0 xuống mức 0 để BẬT sáng dấu chấm tại LED thứ 2. LCALL DELAY_LED và MOV P2, #00H: • LCALL DELAY_LED: Gọi hàm tạo trễ, giúp LED hiện tại sáng đủ lâu để lưu ảnh trên võng mạc mắt người. • MOV P2, #00H: Ngay sau
---	---

<pre> SETB P1.0 LCALL DELAY_LED MOV P2, #00H RET </pre>	khi sáng xong, xuất toàn bộ mức 0 ra Port 2 để chốt TẮT dứt khoát con LED đó.
<pre> CHECK_BTN_C: JNB P3.4, WAIT_C RET WAIT_C: LCALL DELAY_DEBOUNCE JB P3.4, END_C MOV A, SEC CJNE A, #99, INC_SEC MOV SEC, #0 SJMP RELEASE_C INC_SEC: INC SEC RELEASE_C: LCALL DISPLAY MOV P3, #0FFH JNB P3.4, RELEASE_C END_C: RET CHECK_BTN_D: JNB P3.5, WAIT_D RET WAIT_D: LCALL DELAY_DEBOUNCE JB P3.5, END_D MOV A, SEC CJNE A, #0, DEC_SEC MOV SEC, #99 SJMP RELEASE_D DEC_SEC: DEC SEC </pre>	JNB P3.4, WAIT_C và RET: Kiểm tra trạng thái nút. - Kiểm tra xem chân P3.4 (Nút C) có được nhấn hay không. - Nếu có, nhảy đến WAIT_C để xử lý. Nếu không, lệnh RET giúp thoát hàm ngay lập tức để tiết kiệm thời gian. LCALL DELAY_DEBOUNCE, JB P3.4, END_C: Chống dôi phím. - Gọi hàm tạo trễ để kiểm tra tín hiệu. - Sau khi trễ, kiểm tra lại. Nếu nút đã nhả ra (mức 1), chương trình hiểu đó là nhiễu và nhảy đến END_C để hủy bỏ thao tác. MOV A, SEC, CJNE A, #99, INC_SEC, MOV SEC, #0: Tăng 1 giây và chống tràn. - Tăng số Giây (SEC) lên 1 đơn vị. - Lệnh CJNE kiểm tra giới hạn: Nếu đang là 99 mà vẫn bấm tăng, nó sẽ đặt số Giây về 0. MOV A, SEC, CJNE A, #0, DEC_SEC, MOV SEC, #99: Giảm 1 giây và chống tràn.

<pre> RELEASE_D: LCALL DISPLAY MOV P3, #0FFH JNB P3.5, RELEASE_D END_D: RET </pre>	- Giảm số Giây đi 1 đơn vị. - Lệnh CJNE kiểm tra giới hạn: Nếu đang là 0 mà vẫn bấm giảm, nó sẽ đặt số Giây lên mức tối đa là 99. RELEASE_C: LCALL DISPLAY, MOV P3, #0FFH, JNB P3.4, RELEASE_C: Chờ nhả phím. - Vòng lặp này giảm vi điều khiển lại cho đến khi bạn thả nút bấm ra, đảm bảo mỗi lần nhấn chỉ tăng/giảm đúng 1 đơn vị. LCALL DISPLAY: Liên tục quét LED trong lúc chờ để màn hình không bị tắt đen khi bạn giữ đè phím. MOV P3, #0FFH: Liên tục ép Port 3 làm chân Input.
<pre> DELAY_LED: MOV R7, #150 DJNZ R7, \$ RET DELAY_DEBOUNCE: MOV R6, #20 DL1: MOV R7, #250 DJNZ R7, \$ DJNZ R6, DL1 RET END </pre>	Các hàm tạo độ trễ cho việc chống nhiễu và quét led.

2. Video mô phỏng hoạt động của mạch:

https://drive.google.com/drive/folders/1Fr2JGH9O_tvIJ-s-BkVImQG1EzHqZggn?usp=drive_link